

e-TASMI SYSTEM ARCHITECTURE REPORT

AI-Powered Quran Learning Platform — Presentation Guide for Lecturer

Project Name: e-Tasmi
Report Type: System Architecture Explanation
Prepared For: Academic Presentation / Viva
Date: June 29, 2026

Table of Contents

1. Introduction
2. What Problem Does e-Tasmi Solve?
3. Overall System Architecture (3 Layers)
4. Main Features and How They Work
5. Data Flow Diagrams (3 Flows)
6. AI Technologies Used
7. Why We Use Different AI Models
8. Prompt Engineering Strategy
9. Security Design
10. Technical Implementation Notes
11. How to Present This to Your Lecturer
12. Conclusion
13. Quick Reference Card

1. Introduction

e-Tasmi is a web-based Quran learning platform that uses **Artificial Intelligence (AI)** to help students practice recitation and get feedback,

while also providing a **text chat assistant** and a **voice assistant** for Quranic learning.

The system is built using:

- **Frontend:** Browser (HTML, CSS, JavaScript — no AI SDK on the client)
- **Backend:** Java 17, Apache Tomcat 9, Servlets
- **AI Provider:** OpenAI REST API (called from the server only)

The architecture diagram shows three horizontal layers:

1. **Client Layer** — what the user sees in the browser
2. **Server Layer** — business logic and AI orchestration
3. **OpenAI REST API Layer** — external AI services

2. What Problem Does e-Tasmi Solve?

Traditional Quran learning requires an instructor to listen to every student recitation and give detailed feedback. This is time-consuming and not always available.

e-Tasmi solves this by:

- Letting students **record their recitation** in the browser
- Using **AI to transcribe** what they said (Speech-to-Text)
- Using **AI to evaluate** their recitation against the expected Quran text
- Providing a **Quran chatbot** for questions about tafsir, tajweed, meaning, etc.
- Providing a **voice assistant** so students can speak and hear answers
- Offering **AI translation** when official translations are unavailable

3. Overall System Architecture (3 Layers)

Layer 1: Client (Browser — Vanilla JavaScript)

This is the user interface. Three main parts:

Component	What It Does
Student Recitation	Uses MediaRecorder to capture audio (webm/opus format) and upload it to the server
Quran Assistant (Text)	Chat UI where the student types questions and receives markdown replies
Voice Assistant	Uses microphone for input and plays back AI-generated speech (MP3)

Important: The browser never talks directly to OpenAI. It only talks to our Java server.

Layer 2: Server (Java 17 · Tomcat 9 · Servlets)

This is the brain of the application.

Servlets (Entry Points)

- **StudentRecitationServlet** — Receives and stores audio files (locally in /uploads or on Cloudinary)
- **InstructorEvaluationServlet** — Triggers the AI analysis when an instructor requests evaluation
- **StudentQuranAssistantServlet** — Handles text chat requests
- **StudentQuranVoiceServlet** — Handles voice assistant requests

Services (Business Logic + AI)

- **RecitationAiAnalysisService** — Core AI pipeline for recitation evaluation
- **QuranAssistantService** — Text-based Quran chatbot logic
- **QuranVoiceService** — Speech-to-text and text-to-speech for voice mode
- **QuranAiTranslationService** — AI translation with caching

Security

- **OPENAI_API_KEY** is stored as a **server-side environment variable**
- The API key is **never exposed** to the browser or client code

Layer 3: OpenAI REST API (External — HTTPS)

Endpoint	Purpose	Model Used
/v1/audio/transcriptions	Convert speech to text	gpt-4o-transcribe
/v1/chat/completions	Reasoning, chat, translation	gpt-4o or gpt-4o-mini
/v1/audio/speech	Convert text to speech	gpt-4o-mini-tts

Note: We do **not** use the OpenAI SDK. We use Java's built-in HttpClient with custom JSON handling.

4. Main Features and How They Work (Step by Step)

Feature A: Recitation Evaluation (Most Important AI Feature)

Who uses it: Student records → Instructor triggers evaluation → Report shown

Step-by-Step Process

1. **Student Records Audio** — Browser uses MediaRecorder to capture voice; audio saved as webm/opus and uploaded.
2. **Server Stores the Audio** — StudentRecitationServlet receives the file; stored in /uploads or Cloudinary.
3. **Instructor Triggers Evaluation** — InstructorEvaluationServlet calls RecitationAiAnalysisService.
4. **AI Step 1: Transcription** — Audio sent to OpenAI /v1/audio/transcriptions using gpt-4o-transcribe. Output: plain text transcript.
5. **AI Step 2: Evaluation** — Transcript + expected Quran passage sent to gpt-4o. AI produces structured JSON report with correct/missing/incorrect/extra words, pronunciation notes, score, summary, and feedback.
6. **Report Displayed** — JSON report saved and rendered in evaluations.jsp for instructor and student review.

Fallback handling: Empty audio → Cannot evaluate | Not Quran (high confidence) → Rejected | AI fails → Local word-diff fallback used.

Feature B: Quran Text Assistant (Chatbot)

1. Student types a message in the chat UI
2. Browser sends message to StudentQuranAssistantServlet
3. Server calls QuranAssistantService
4. Service sends message to OpenAI /v1/chat/completions (gpt-4o-mini)
5. AI reply restricted to Quranic learning topics only
6. Markdown reply displayed in the chat

Feature C: Voice Assistant

1. Student speaks into the microphone
2. Browser sends audio to StudentQuranVoiceServlet

3. **Speech-to-Text:** QuranVoiceService.transcribe() → gpt-4o-transcribe
4. **Chat Reasoning:** Transcript → QuranAssistantService → gpt-4o-mini
5. **Text-to-Speech:** Reply → QuranVoiceService.synthesize() → gpt-4o-mini-tts
6. MP3 audio (base64) sent back; browser plays the spoken answer

Feature D: AI Translation

1. Student requests translation for a verse
2. Server checks Quran Library for official translation
3. **Cache check:** If already translated → return cached result (no API call)
4. **Cache miss:** QuranAiTranslationService → gpt-4o-mini
5. Translation cached and returned (labelled as "AI translation")

5. Data Flow Diagrams (3 Flows)

Flow 1: Recitation Evaluation (Orange)

```
Student uploads audio
→ Server stores audio
→ OpenAI transcribes audio (STT)
→ OpenAI evaluates recitation (GPT-4o)
→ JSON report generated
→ Displayed in browser
```

Flow 2: Chatbot + Voice (Blue/Teal)

```
Text: User message → QuranAssistantService → gpt-4o-mini → Reply
to UI
```

```
Voice: Mic audio → STT → QuranAssistantService → gpt-4o-mini →
TTS → MP3 to browser
```

Flow 3: Translation (Green)

```
Quran Library → Cache check → (if miss) gpt-4o-mini → Translation
to user
```

6. AI Technologies Used

Technology	How We Use It
Large Language Models (LLM)	Chat, evaluation reasoning, translation
Generative AI	Producing feedback reports and chat replies
Natural Language Processing (NLP)	Understanding student questions and Quran text
Automatic Speech Recognition (ASR)	Converting recitation audio to text
Text-to-Speech (TTS)	Converting AI replies to spoken audio
Prompt Engineering	Designing instructions for reliable, structured output
Semantic Analysis	Comparing recited text vs expected Quran passage
Transformer-based Models	Underlying architecture of GPT-4o models

7. Why We Use Different AI Models

Model	Used For	Reason
gpt-4o-transcribe	Speech-to-text	Better accuracy on classical/Quranic Arabic
gpt-4o	Recitation evaluation	Strong reasoning to compare text and judge mistakes
gpt-4o-mini	Chat, translation	Faster, cheaper, good for conversational tasks
gpt-4o-mini-tts	Voice output	Low-latency text-to-speech

Simple explanation: We use the powerful model (GPT-4o) only where deep reasoning is needed — evaluation. For chat and translation, we use the lighter model (GPT-4o-mini) to save cost and improve speed.

8. Prompt Engineering Strategy

- **Zero-Guessing Contract** — AI must not invent mistakes; only report what it can verify.
- **JSON Output Schema** — Evaluation returns structured JSON, not free text.
- **Deterministic Evaluation** — Temperature = 0 and fixed seed for consistent, repeatable results.
- **Scope Restriction** — Chatbot limited to Quranic learning topics only.
- **Single-Pass Evaluation** — Transcript + expected passage in one GPT call to reduce errors.

9. Security Design

Security Measure	Explanation
Server-side API key	OPENAI_API_KEY stored in environment variable on server
No client exposure	Browser never sees or sends the API key
HTTPS communication	All OpenAI calls use secure HTTPS
Servlet authentication	Only logged-in users can access AI features

10. Technical Implementation Notes

1. **No OpenAI SDK** — Raw `java.net.http.HttpClient` for full control
2. **Custom JSON parser** — Lightweight `MiniJson` utility
3. **Multipart upload** — Audio sent as `multipart/form-data`
4. **Caching** — Translations cached in memory
5. **Fallback logic** — Local word-diff if AI fails
6. **File size limits** — Max 25MB; minimum ~3 seconds audio
7. **Timeout handling** — 90-second request timeout for AI calls

11. How to Present This to Your Lecturer (Step-by-Step Script)

Opening (30 seconds)

"Assalamualaikum / Good morning. My project is e-Tasmi, an AI-powered Quran learning platform. It helps students practice recitation with automated feedback, and includes a Quran chatbot and voice assistant."

Part 1: Show the Architecture Diagram (1 minute)

"The system has three layers. At the top is the browser where students interact. In the middle is our Java server that handles all logic. At the bottom is the OpenAI API that provides AI capabilities. The browser never talks to OpenAI directly — everything goes through our secure server."

Part 2: Explain Recitation Evaluation (2 minutes)

"The main AI feature is recitation evaluation. First, the student records their voice in the browser. The audio is uploaded to our server. When the instructor triggers evaluation, we run a two-step AI pipeline: Step 1 — send audio to OpenAI for speech-to-text using gpt-4o-transcribe. Step 2 — send the transcript plus the expected Quran text to GPT-4o for reasoning. The AI compares what was recited against what should have been recited, and returns a structured JSON report with mistakes, score, and feedback. We use temperature zero and a fixed seed so the evaluation is consistent every time."

Part 3: Explain Chatbot and Voice (1 minute)

"We also have a Quran assistant chatbot powered by GPT-4o-mini. Students can ask about tafsir, tajweed, or memorization. It is restricted to Quranic topics only. The voice assistant adds speech-to-text and text-to-speech, so students can speak their question and hear the answer."

Part 4: Explain Translation (30 seconds)

"For the Quran library, if an official translation is missing, we use AI translation with caching — once a verse is translated, we store it so we do not call the API again."

Part 5: Explain AI Technologies (1 minute)

"This project uses several AI technologies: LLM for reasoning and chat, ASR for speech recognition, TTS for voice output, and prompt engineering to control output quality. We chose different models for different tasks — GPT-4o for evaluation because it needs strong reasoning, and GPT-4o-mini for chat because it is faster and cheaper."

Part 6: Explain Security (30 seconds)

"Security is important — the OpenAI API key is stored only on the server as an environment variable. It is never exposed to the client browser."

Closing (30 seconds)

"In summary, e-Tasmi combines traditional Quran learning with modern AI to give students instant feedback on recitation, plus an intelligent Quran learning assistant. Thank you."

12. Conclusion

e-Tasmi demonstrates a practical integration of AI into an educational web application:

- **Recitation evaluation** uses a two-step AI pipeline (transcription + reasoning)
- **Chat and voice assistants** provide interactive Quranic learning support
- **AI translation** fills gaps when official translations are unavailable
- **Security** is maintained by keeping all AI calls server-side
- **Prompt engineering** ensures reliable, structured, and consistent AI outputs

The architecture is **modular, secure, and cost-efficient**.

13. Quick Reference Card (For Viva Questions)

Question	Short Answer
What AI models do you use?	gpt-4o-transcribe, gpt-4o, gpt-4o-mini, gpt-4o-mini-tts
How does evaluation work?	Audio → STT → Compare with expected text via GPT-4o → JSON report
Why GPT-4o for evaluation?	Needs strong reasoning to judge recitation accurately
Why GPT-4o-mini for chat?	Faster, cheaper, sufficient for conversational tasks
How is API key secured?	Server environment variable, never sent to browser
What is prompt engineering?	Designing AI instructions for reliable structured output
What is temperature = 0?	Removes randomness for consistent evaluation results
Do you use OpenAI SDK?	No — raw Java HttpClient with custom JSON handling

END OF REPORT